

Pre-Entry Preparation Assignment Deliverables:

”AISIMS Data Model – Working Log”

AISIMS & AIVISTA

Assignee: Daffa

Deliverable Period: Week 3

1. Assignment Summary and Reflection

i. My Approach to the Week 3 Assignment

The Week 3 Assignment is about further expanding the architecture foundation built in the previous week, implementing actual business logic into the software. I spent this week mostly on system-design brainstorming, with AI used mainly as a judgement assistant to foresee issues with approaches I came up with along the way, to create visualizations for each scenario, and to look up best practices that support my decisions.

As for coding work, I still relied heavily on Codex, although each commit is specified based on the workflow I generated so that supervision could be done properly. My approach this week was to let Codex do the implementation while I reviewed and read the code, since in Week 2, building everything from the ground up myself without knowing the outcome in advance turned out to be difficult, scattered, and prone to pitfalls. What I did instead was feed the AI a per-commit workflow matching the final design, then review, learn from, revise, and submit the result. Eventually, there was still back-and-forth between revision and identity design, the ERD, and the codebase update, as problems surfaced while working on any one of the three, but as a starting principle, I believe it was correct to first establish how data is handled, then the data model, and only then implement it into code.

The summary of items I delivered this week includes:

Product	Items
Written deliverables	• ERD of all entities, fields, and key relationships, including supporting entities tied to revision logic outside the data hierarchy mentioned in the pack, along with its validation rules • Write-ups on geometry notes in compliance with the latest design decisions • Design decisions on revision handling and the identity model
Codebase update	• Established an updated model built on the updated ERD • Establishing a service layer module for validation and revision handling on the backend • POST and GET endpoints for inputting revision and displaying fed data

Product	Items
	• Simple read-only frontend displaying the database entries
Miscellaneous	• Revision and identity model presentation • Outstanding questions • Working Logs • Updated preparation schedule

When reviewing the preparation schedule, I realized that part of the Week 4 assignment is present in this week's design, including validation logic test cases and an implemented service model. The decision to bring these forwards was intentional, this simply because designing validation rules without a working input pipeline to run them against makes it very difficult to visualize revision behavior or validate that the workflows actually work as intended.

ii. Knowledge Checks

Questions needed to be answered without the help of AI ahead of next week's assignment. AI was still used for grammar checks but did not provide any context or idea for the answer.

1. Why can B1.02.A1A2 be a useful member code but a poor stable primary identity?

The distinction lies in readable versus traceable identity: "B1.02.A1A2" is a useful member code for readability because of its human-friendly nature; it instantly tells the section, floor, and the location of a member. However, for this same reason it is also a poor traceable id, because revision for a specific member could target one of the qualities the readable id consists of (section and location) throughout the project lifecycle, hence a non-stable identity for a database. Therefore, to accompany a "readable" identity, there has to be a primary "traceable" key tied to it, which is unique and not necessarily human-readable.

2. What should happen to identity if a member changes section only? What if it moves?

Readable identity should serve the purpose of informativeness and currency, therefore it should reflect any change to that member, whether a section change or a move. However, if it belongs to an existing member, the stable ID cannot change, since this is what determines whether a member is "changed" or "added." A changed member with the same stable ID can be compared across revisions, whereas a new member has no previous stable ID against which its change status could be derived. This is what underlies the change-status workflow in the identity codebase design.

3. Why can IFC Global Id and AICONS internal ID be separate?

Both IDs serve the same purpose: a stable, non-changing ID representing a member throughout its lifecycle. However, the main difference lies in their source: the IFC ID is a product of BIM export, while the AICONS internal ID is self-assigned.

4. Should beam/column geometry and wall/slab geometry be represented identically?

No, it should not. The categorization being that beam/column is a "member" object which is one dimensional (linear, only has length when section is ignored) whereas walls and slabs is a "shell" object which is two dimensional (polygonal, has width and length when thickness is ignored). Therefore, for the same "object" field, objects have different geometry representation.

5. Why separate Material, Section/Dimension, and Structural Member?

Material and section feed structural members through a reusable reference relationship. Meaning that many members could reuse the same qualities described in the catalog of material and section. It is a means of building an effective database architecture. Instead of writing all the fields and qualities (e.g. material properties and section dimensions) to every single member entry, a database could store the fields as separate neat catalog which recalled by using foreign key field in the form of ID, so that one field can represent the whole table of qualities the material/section possess

6. What problems occur if Zone and Storey relationships are inconsistent?

Zone and storeys/floors represent the location or area in which an object resides. These fields need to be consistent in terms of which building they represent and the locational quality they tie to. Any inconsistency means the objects residing within it also implicitly undergo an unintentional change (e.g. a floor's height or name is edited so that it represents another floor in another building, causing objects to move entirely along with the edited floor). Although the model designed in this training proposal does not tie the locational qualities of the floor/zone fields into the object's own locational fields (a member's location is represented solely by its geometry points), inconsistencies in the floor and zone relationship would still cause object-sorting problems or drawing-production mismatches: when sorting which building and floor contains which objects, the picture becomes unclear, since a member could effectively appear to move between buildings, and once this reaches drawing production through the output funnel, the floor level and the object's actual level could end up misaligned.

7. When is JSONB appropriate, and when is a separate table better?

JSONB is appropriate when an entry is logically a bundle, or when a field's shape genuinely varies. For example, an object's geometry points: the object type dictates how many points the entry needs (a one-dimensional object requires two points, whereas a polygonal object requires three or more), and the logical constraint on the field's size is loose, but the values still belong to the same single attribute. A separate table becomes the better choice once an attribute needs its own identity, relationships, or independent querying, rather than being read only as a whole alongside its parent row.

(This question raises a concern about the fragility of the dimension field, which is another polymorphic entity handled in JSONB. Revising the data model for this was initially considered, but for now it is addressed instead in the Acknowledgement of Limitations, with an explanation of why I believe a revision is still needed here.)

8. How should Reinforcement and Quantity connect to Structural Member?

One way to connect a structural object (member or shell) is to tie these fields into the same table, so that one entry represents an object with all of its constituent configurations, from location information, sections, and materials, down to reinforcement and calculated quantity. However, as the logic dictates, one structural object could have more than one reinforcement entry by bar role, as stated in the first week's supplementary pack. These bar roles each describe a specific reinforcement location within an

object, each with its own quantity. Enforcing reinforcement and quantity into designated fields on the object itself would cause a very wide table, which is not ideal, since it treats something that should be a set of entries as a set of columns instead. A neater approach is to tie a structural member's ID to a one-to-many relationship with its reinforcement, where each bar role can be represented and detailed; quantity is then called with the same key to summarize the total calculated volume from the rebar, as well as the volume of the section itself.

9. Why should earlier revision information not simply be overwritten?

As the first pack stated, the way AICONS handle revision is as a snapshot rather than an edit, therefore every historical evidence is treated as archive. This is actually beneficial to the practice in the construction project field as the most important data is not only the latest but also how that latest version got there chronologically

10. What AI suggestion about your Week 3 model did you reject or modify, and why?

The use of AI in design for this week is mostly to weigh and support decisions on any possible crossroads emerging from my end. However, some suggestion generated from AI is also present, some of which I decline, some major crossroads are as follows:

Product	AI Suggestion	Rejection	Documentation
Grid entity model	Grid coordinates stored as an implicit scalar (e.g. {"value": 9000}), with axis meaning inferred from the field <i>grid_axis</i>	Changed to an explicit {x, y} object regardless of the <i>grid_axis</i> entry, so the frontend does not need conditional logic to plot a point	Commit hash 3af0734
Input model	AI suggested keeping input fields in the pydantic class as raw IDs (floor_id, zone_id, sect_id, mat_id), reasoning that API-layer inputs should stay on stable identifiers	Input was changed to using its contextual, user-facing identifiers (floor names, zone labels, section labels, material names, and bar's diameter, type and grade), while the data storing to their respective database-level stable id's resolved server-side. A structural engineer will never manually look up an opaque database row ID where the natural engineering property (a floor's name, a section's label) is what's on hand.	Commit hash db23d57
Identity convention	Entities' ID only had a database-facing ID's without accompanied by user-facing labels and mostly was hardcoded names (e.g. rebar catalogue database ID was set to its label instead of algorithmic-	I forced a convention that all entities must have an autogenerated database row identifier (-001, -002, -003, etc.) for keying purposes and only then supplemented with its context as its readable labels. The reason being that there must be a distinction between a database indexing identities and a legible, context-filled	Commit hash 4d59ba8 ; c778b8b

	auto generated number)	namings. This is the same principle with stable id versus member mark for structural objects. Confusing between the two will either confuse users with misleading name (e.g. id REV-002 turns out to be a first revision on the second building instead of a second revision of the first building) or overcomplicate the database-level management	
Identity convention	All entities' ID were generated with UUID4 strings	Changed to a simple naming convention with 3-4 letter abbreviation of the entity name followed by 3-4 digits of the sequence of the entry created (e.g. PRJ-001, BLD-001, RBAR-0001 etc). While uniqueness and autogenerative names for the database-facing identities was the goal, it also still required to be readable by developer or system engineer working with the database, especially at this stage where data handling is still done from the backend through /docs and on the server-side through psql	Commit hash 9ce8b18

iii. Acknowledgement of Limitations

a. Fragile polymorphic entity handling for section dimension field

When working on the knowledge check, the codebase was ready to be wrapped up as it already ran as intended, albeit with its limitations of scope and minor bug. However, when stumbling upon the JSONB question, it raised a reflection on the model of the “dimension” field under section entity. Handling the shape, width, and height depending on the section model and leaving it to the correct input felt very fragile as opposed to making separate field in which the shape, width, height, and thickness is explicitly described in the database.

While reviewing the quantity calculation algorithm inside the service model (revision service), I noticed that the calculation is heavily dependent on the JSONB string input; validation rules are simply inconsistent to enforce on an additional, unexpected property.

The decision sits at a crossroads between keeping the field as JSONB input with validation enforced on the input side, or having separate fields for all dimensional attributes (e.g. section.shape, section.width, section.height, etc). A third option, an entirely separate table for each dimension attribute, was promptly eliminated, since it would require too many entities. JSONB was kept because both the shape vocabulary (how stairs and footings should be represented) and the enforcement mechanism (service-model validation vs. database-level CHECK constraint) are still undecided, and JSONB lets that decision stay open without locking in a schema shape prematurely

The concern with staying on JSONB is that the current service-model validation, unlike a CHECK constraint, does not yet reject an unexpected extra property (e.g. a circular section that also carries a width field), which remains the fragility mentioned above. A final decision has not yet been made, since how stairs and footings will be handled is still unresolved.

b. Ambiguity for stairs and footings objects model

In the current model, I purposely left the dimension validation behavior for stairs and footings untouched, since I have not yet confirmed how to handle these objects: whether to treat a stair as a cross-floor diagonal slab, whether to describe tread and riser in the database, how to handle footings of different shapes, and so on.

c. Input funnell through /docs requires user to type in the database-facing ID's instead of its contextual names

In my identity design, I purposely intended every human input to reference entities by their contextual names rather than their auto-generated database row IDs, which is why every Pydantic ObjectInput field was changed to accept names. However, the endpoint path parameters in /docs still require the user to input entities by their database ID, requiring the user to look up the database or the frontend list for the intended field. Once a UI is built and input is handled through the user's own navigation and clicks, this will no longer be a problem, and the input funnel will work as designed.

iv. Topics Understood and Proposed but Not Yet Fully Designed and Built

Several pending topics to the system that have not been fully designed because certain open questions regarding how it was handled or scope limitations of this week's assignment:

a. Deletion Workflow for Project Configuration

Project Configuration entities (Material, Section, Bar Spec, Floor, Zone, Grid, Building, Project) currently have no DELETE endpoint. Removing a row requires going directly through SQL to exclude from the scope of the Week 3 requirements. A future DELETE would need to check for existing references first and reject if any exist, per the design sketched during this phase.

b. Object History View

Currently there is no endpoint or UI that surfaces a single object's full history as a timeline. To see how one member changed across revisions, someone would have to manually cross-reference multiple revision GET endpoints by its stable ID. However, the current data itself already supports this fully, every revision's row for the same object is linked by stable id with its respective change_status recording what happened at each point. Therefore, this remains a presentation layer gap. A future endpoint could filter GET endpoint by stable id across every revision of a building to produce this view directly.

c. Full UI-Driven Interaction for Revision Input

Revision triggers happen exclusively through a JSON body via /docs. This is presently not actually convenient for an engineer in practice: even though input is name-based rather than ID-based, someone typing the JSON by hand still has to get every field entry name exactly right with no autocomplete, no dropdown, and no protection against a typo silently failing the validation workflow. The container for the further-developed version is already

in place structurally, name-based fields are exactly what a future frontend form would need to offer as click-to-select pickers, backed directly by the GET endpoints already built. Once that frontend exists, the same input schema becomes genuinely engineer-friendly.

d. Engineering Scope Validation

Current validation workflow, whether the one built inside the service model or in the Pydantic model, currently only touches system-level validation (e.g. data types, non-nulls, and the like) and a few surface-level engineering-logic checks (object geometry, section dimension logic, and similar). Civil-engineering-specific validation or warning messages, such as missing objects, floating members, or anything else that would not physically make sense in a building, are not yet designed. This is currently considered a validation step happening upstream, performed by the structural engineers themselves.

e. Staging Area Ahead of Committing Input and Triggering Revision

Input or the revision cycle is designed to be expensive, meaning a revision should not be triggered by everyone or at every moment, since it is crucial for the project; yet the current input is very straightforward and frictionless. It is suggested that before committing any input, ingested data should pass through a “staging” step, similar to a Git workflow, where the difference between the current and previous revision could be checked and approved, especially for bulk data input.

2. AI-assisted Working Log

This section explains the breakdown building up to the final design write-ups in the Written Deliverables document. Before reading this section, it is suggested to read the final Written Deliverables document first for more context, since not everything here is mentioned in chronological order.

i. Pre-Implementation Works (Pack Review and Written Design)

9 September 2026 — Supplementary Pack Review and Specifying Deliverable Goals

Context

I started by reviewing the pack, both the previous weeks' and this one's. I noted areas I had missed and formulated the deliverables into actionable smaller tasks, dividing them into codebase updates, written deliverables, and miscellaneous deliverables. AI was asked to help formulate summary documents and cross-check that everything required, and everything that needed caution, was met and understood.

Prompt (verbatim, translated to English)

Can you turn this into a simple to-do list, an update from what we just discussed? Some of it already overlaps with the previous list, but some things still need to be added. Refer to the document and get back to me.

Make it two sections:

- a to-do list per deliverable header (writing, presentation, codebase update, etc.)
- things I need to be cautious about

Let me review it first; if it's good, next I want to turn it into a PDF.

Result

The result was a summary document listing the crucial points and list of things to be done in Week 3, along with a cross-reference to the original document.

Judgement and Validation

The summary was checked against the supplementary pack and used as a basis for task breakdown. In summary, the classification as mentioned in the context (codebase, written, and misc.) consists of these following items.

Deliverables	Items	Format
1. Codebase update	a. Week 3 branch and Git evidence b. New database entities based on project-down-to-revision hierarchy c. Validation rules as database constraints In models.py d. Identity model design implemented into database	A GitHub repository
2. Written deliverables	a. ERD (non-assisted) b. Data Dictionary c. Key relationship description d. Geometry representation notes e. Identification model design f. Revision model design g. Validation rules	One document
3. Miscellaneous deliverables	a. Short presentation for point 2e and 2f b. Outstanding question update with status c. Schedule update d. Working and AI Usage log (this document) with answers to the questions in Section 11 of the pack	Four separate documents

My pre-revision Week 1 assignment left out a lot of what was actually asked, and my Week 2 assignment took considerable effort to finish because I did not yet know the right approach. Therefore, in Week 3, I started by strategizing the output, breaking it down, and classifying it. By adhering to this list, my work could stay focused instead of going in every direction, hopefully avoiding what happened last week, where I wandered off-track and burned time on the deliverables; this way, my work could be more strategic.

9-11,14 September 2026 — Designing Identity Model and Revision Handling

Context

Identity Model Design – In understanding the identity model design, I first confirmed the difference between “readable” ID and “traceable” ID. What I synthesized back in Week 2 was the former, where an engineering identifier such as “C1.02.A1B1” can easily be interpreted by engineers in the field. However, this ID poses a database problem, since one member can end up with a changing identity across its revisions. Therefore, a stable, non-changing identity representing a single member across revisions throughout the whole life of a project is needed; I later learned the term “Stable ID.” Expanding on that same logic, the IFC Global ID serves the same purpose: the Stable ID is tied to an internal system, while the IFC ID depends on external BIM software. Further into the design process, I also realized this logic applies to every entity present in the system, not just structural objects, so I went on to design accordingly: every entity should have a stable, identifying database index and a contextual mark (e.g. material stable ID

MAT-001 is accompanied by its contextual name “K250,” alongside other descriptive fields inside the database).

AI was then used to help weigh judgement on the system design:

1. Clarifying the workflow of “Stable ID registration” of a new member versus “Stable ID self-check” for existing member that has been changed in a revision
2. Clarifying field uniqueness requirement (for constructing the ERD)
3. Cross-checking database relationship dependencies (confirming an entry called by its contextual name can refer to its stable ID in any case: input, revision, and GET endpoints)

Revision Model Design – Identity definition is then carried forward into designing the revision model. How a member is determined to be changed, added, unchanged, or deleted between revisions is keyed on these identifiers. The revision model's decision points revolve around this identifier; although not mentioned in the pack, visualizing revision behavior becomes very difficult without the input behavior in mind, so input behavior was also considered while designing the revision model.

AI was then used to help weigh judgement on the system design:

1. Which entities are subject to frequent/seldom revision
2. What possible changes are exposed to each entity
3. The required validation for each change
4. Whether or not all changes trigger a revision cycle
5. Determining how a revision version handling should be treated: tied to the member, the whole project, or a combination of the two
6. Determining whether a deleted member is implicitly absent in the database or explicitly mentioned, and clarifying validations for deleted member
7. Determining whether a resurfaced deleted member gets a restored status or added status, as well as designing the validation workflow that enables it
8. Designing service model that triggers every time a revision is uploaded

Input Design – Revision is triggered every time new “input” is carried into the system, so designing input behavior is essential. At this stage, input design is limited to which data is entered by a human versus which is system-generated or system-assigned, and to understanding how a single entity can contain both. This stage leaves out UI behavior: put simply, this week's design task was about determining what enters the system and how, what must be entered by a human and in what shape, and how input differs between project configuration and project design.

AI was then used to help weigh judgement on the system design:

1. Differentiating between “project configuration” inputs and “project design” inputs and how each group is stored
2. Choosing between a full bulk-upload revision and a lighter, interactive edit mode

Result, Judgement and Validation

Assistance on design decisions is based on scanning the supplementary packs uploaded to the AI's project context, to check whether a company preference is already present in any of the provided documents, and by foreseeing issues in implementing any decision through simple generated visual cases. The final decision, however, still rests on my own judgement. Below is the list of decision crossroads that were then decided with AI's help; the prompts used depend on the context being discussed, but in summary, my

prompts usually asked for judgement and a scan of foreseeable consequences. Note that all these decisions are training-proposal only.

a. Field Names and Terminologies

	Item	Decision	Explanation
D01	Changing entity name “storeys” to “floors”	Change and update all the existing wording in foreign keys, endpoints, and seed python files	Matching the wording used in the source pack documents
D02	Changing entity name “dimensions” to “sections”	Change and update all the existing wording in foreign keys, endpoints, and seed python files	Avoids confusion with the “dimension table” concept from data warehousing terminology
D03	Changing entity name “members” to “objects”	Change and update all the existing wording in foreign keys, endpoints, and seed python files	Structural object such as slabs and walls are considered shells rather than members

b. Identity Model

	Item	Decision	Explanation
D04	Separate “identity” table vs direct column on object entity	Separate table	change_status and active_status workflow depends on identity being tracked independently from per-revision fact rows
D05	Stable ID assignment when IFC GlobalId is absent	Decided to be manual human-declared input (via an <i>is_new</i> input flag on the model)	Without an external IFC identifier to match against, there is no way for the system to determine automatically whether an uploaded object is new or a continuation of an existing one
D06	Duplicate/error detection during stable_id assignment	Decided to be out of scope for this assignment. is treated as predetermined and correct by assumption	Manual stable_id assignment is understood to be prone to human error, however a system check would require fuzzy-matching or other probabilistic logic
D07	Types of identifiers	All entities are represented by database-facing *_id and user-facing *_name/*_label/*_mark contextual identifiers. *_id are non-nullable and uniquely autogenerated sequentially whereas the contextual identifiers follow the convention of the user	*_id is intended as stable identifier for database row, whereas *_name/*_label/*_mark are reserved for engineer-view
D08	Autogenerated stable ID	For objects: using UUID4, generation of new string code is triggered through the input’s	The database record has to be unique and error-proof but still needs to be readable by the developers maintaining it, for objects its

conventions	is_new flag For other entities: generated using sequential entries such as PRJ-001, BLD-001, RBAR-0001	purpose is to be an internal, opaque, revision-proof anchor, not a human-facing reference.
-------------	---	--

c. Input Architecture (Made ahead as part of designing the Revision Model)

Item	Decision	Explanation
D09 Data input grouping	Data inputs are split into two groups: <i>Project Configuration</i> and <i>Project Design</i>	<i>Project Configuration</i> (Building, Floor, Zone, Material, Section, BarSpec, Grid) is set up independently, ahead of any revision cycle. <i>Project Design</i> (Object, Reinforcement, and derived output like Quantity) flows through the revision workflow. Each has its own designated POST endpoint.
D10 Data input funnels	<i>Project Configuration</i> : separate POST endpoints for each data inputs <i>Project Design</i> : two endpoints with different behaviour, bulk upload and interactive edit	<i>Project Configuration</i> : every project setups can be user-defined ahead of inputting objects into the project <i>Project Design</i> : bulk uploads define the entire batch of data for a revision version whereas interactive edit only alters the intended objects and carry forward the untouched object
D11 Human-friendly input	Input models on POST endpoints are referred to its contextual identity instead of their database-facing ones	A structural engineer will never manually look up an opaque database row ID to know which floor or section they mean where the natural engineering property (a floor's name, a section's label) is what's actually on hand
D11 Delete behavior for each data input funnels	<i>Project Configuration</i> : can't be deleted when the configuration is used / referred in an object entry (append-only rule) <i>Project Design</i> : in bulk revision, absent data which previously exist is considered to be an omission whereas in interactive edit, deleted data is explicitly chosen / described	<i>Project Configuration</i> : Can't be deleted when currently referred in an object revision, unless the entire reference is deleted, a project configuration behave as an append-only entities <i>Project Design</i> : Bulk uploads define the entire batch of data for a revision version whereas interactive edit only alters the intended objects and carry forward the untouched object
D12 Append-only inputs vs changeable data for <i>Project Configuration</i>	Changeable data are applied for higher hierarchy Project Configuration data such as project name or building name, lower hierarchy entries that feeds object attributes such as materials can't	Lower <i>Project Configuration</i> change leads to implicit change to the previous revisions, instead of changing a configuration through the primary keys, the user should make a new configuration entry with the intended change and assign the new entry to the newer version

	be changed and therefore an append-only	of the same object
D13	Data uniqueness Uniqueness levels can be categorized as follow (not a determined terminology by design, descriptive only): • <i>Global-scope uniqueness</i> • <i>Building-scoped uniqueness</i> • <i>Composite uniqueness</i> • <i>Non-unique</i> (Refer to the ERD in the written deliverable for more detailed context)	<i>Global-scope uniqueness</i> (project_name, building_name, sect_label, mat_name, barspec_label) for identifiers acting as cross-building lookup keys, so a name alone resolves to exactly one row regardless of which building or project references it. <i>Building-scoped uniqueness</i> (floor_name, zone_label, pour_seq) for identifiers whose meaning is only local, since two different buildings may legitimately reuse the same label (e.g. both having a "Level 2" or "Zone A") without ambiguity. <i>Composite uniqueness</i> (grid_label + grid_axis + building_id, rev_number + building_id) for identifiers that are only unique in combination, not individually. <i>Non-unique</i> For descriptive or value-only fields (obj_mark, elevation, dimension contents, material properties on their own), since duplication across rows is expected and doesn't threaten data integrity.

d. Revision Model

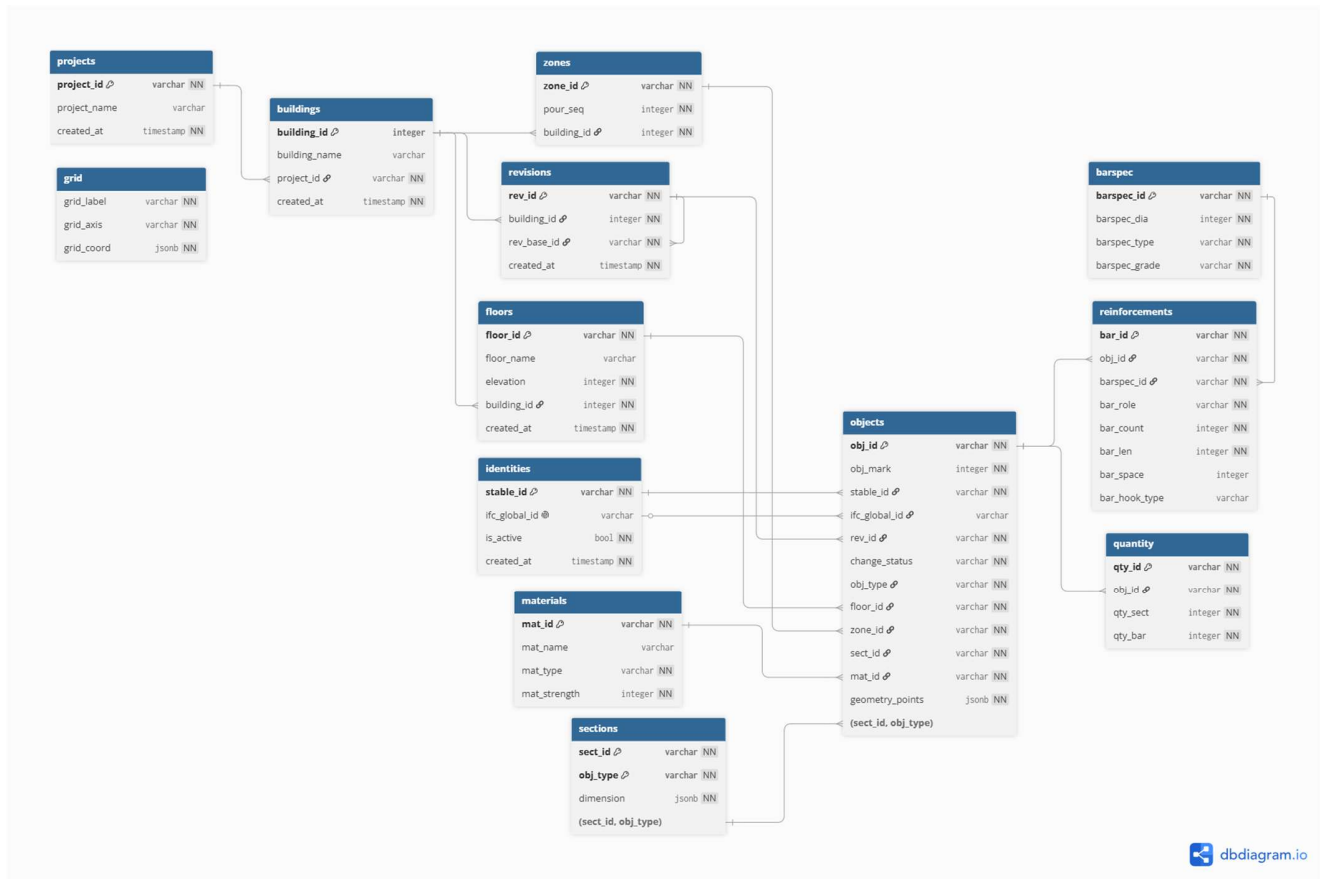
Item	Crossroad	Decision	Explanation
D14	Which data triggers a revision	Derived from the input architecture: only project design group (objects and reinforcements) flows through revision workflow.	Adding a new floor, zone, or object attributes (project configuration data) does not trigger a revision, only when assigning those configurations into an existing or a new object does
D15	Where a revision mark should be tied to: project, or objects	A combination of the two: revision version tied to the entire project and a "change status" tied to the objects to retain snapshot revision principle while maintaining granularity	Snapshot-style revision should rebuild the entire version tied to the whole project even when revision only touch a single object, "change_status" field tied to the singular objects help with granularity so that change identification can be done easily
D16	Linear revision vs branching	Revision is decided to be linear, revision numbering is auto-	Linearity means Revision 3 can not be built based on Revision 1 and should always be based on the previous revision to preserve the chronological

	revision	generated and sequential	order or the project data, based on practical cases
D17	Which revision counts as “current”	The “current” revision is always the latest, as the consequence of D8	Revision number reflects the latest decision happening on the field, should there be any decision to “return” to any previous state in the project, it will count as another revision cycle, not a rollback
D18	Workflows triggered by a revision	1. <i>Pre-validation Gate</i> 2. <i>Stable ID Assignment</i> 3. <i>Change Status Determination</i> 4. <i>Active Status Sync</i> (Refer to the written deliverable for more detailed context)	1. <i>Pre-validation Gate:</i> Before objects can further be processed down the revision model workflow, it undergoes two validation: Referential Check: all reference inside the objects attributes must already have an entry on the <i>Project Configuration</i> Contextual Completeness: all entries have to pass the logical validation rules (geometry definitions, section dimension assignments, etc) 2. <i>Stable ID Assignment</i> Generating a new Stable ID and feeding it to the identity table where is_new returns true or flipping the active status to true where is_new returns false 3. <i>Change Status Assignment:</i> Checking every object entries’ attribute columns and associated reinforcement rows, classifying each as <i>added</i>, <i>modified</i>, <i>unchanged</i>, or <i>returned</i> in comparison with the previous revision's version. 4. <i>Active Status Sync:</i> Checking absent stable ID in the current batch against the previous, flipping active status to false when absent (preventing change_status to return deleted status repeatedly)
D19	Input/revision handling on reinforcement	Reinforcement is designed to have its separate table, revision on reinforcements are tied to its object and does not carry its own change_status or stable ID.	Meaning no revision can't be triggered from changing only the reinforcement table, however, user has to submit the same object through the data input funnel with the revised reinforcement. However, the Input model contains the possibility to seed multiple rows into the same object entry

14 September 2026 — ERD Initial Draft

Context

First draft ERD v.0.1 produced after cohesive design on entities and fields are fixed based on identity model and revision handling. I use dbdiagram.io as the modeling tool using dbml language to determine the relationship and requirement of each field in every entity based on the design with no help of AI.



Initial ERD Version

Prompt (verbatim, translated to English)

Now I'm on the ERD. I've written this DBML, this is ERD v0.1 — check the relationships and the tables.

Result, Judgement and Validation

The resulting judgements were cross-checked against the intended design at that point in the design's version history, and changes were made as follows:

ERD Version History — Issues Found and Changes Made

v0.1 → v0.2

- *rev_base_id* (initially a field where a revision can be branched from a base version, later omitted) was incorrectly non-nullable; made nullable
- *ifc_global_id* was duplicated on *objects*, creating two sources of truth with *identities.ifc_global_id*; removed from *objects*
- *obj_mark* was typed as integer, which cannot hold an alphanumeric mark like "C1.02.A1A2"; changed to varchar

- *grid* had no primary key and no *building_id*, so a grid row could not be scoped to a building or uniquely referenced; both added
- *building_id* was typed as integer while every other identifier in the schema was varchar; changed to varchar everywhere it appears
- Several tables (*materials*, *zones*, *sections*, *barspec*, *grid*) were missing *created_at*, breaking the historic-record convention used elsewhere; added

14-16 September 2026 — Finalizing ERD and Cross-checks

Context

After designing how revision works, which eventually extended into input design, I was able to finalize the entities and fields and review their relationships as well, such as object and section, which I renamed to avoid confusion, or floor, grid, and zone, whose relationship I also changed to be more logical: what was initially meant to feed the member mark became a standalone entity used only for drawing output data. AI was used here to review and help resolve issues by looking up practices from applications like ETABS or IFC export to settle the design decision on the grid-to-floor relationship. The process of building the ERD eventually became iterative, following the problems I ran into while forming a complete design decision, with prompts used gradually along the way to recheck the ERD for errors each time. At this stage, Codebase Phase I was also being built in parallel, and problems found while testing the system fed back into further ERD updates.

Result, Judgement and Validation

The historical changes and updates to the ERD are as follows:

v0.2 → v0.3

- *revisions.rev_base_id*, a self-referencing foreign key meant to enforce linear revision history, was found to only validate that its target exists, not that the target is the latest revision, so it did not actually prevent branching; removed entirely and replaced with *rev_number* (integer), computed rather than stored as a free choice, enforced unique per *building_id*

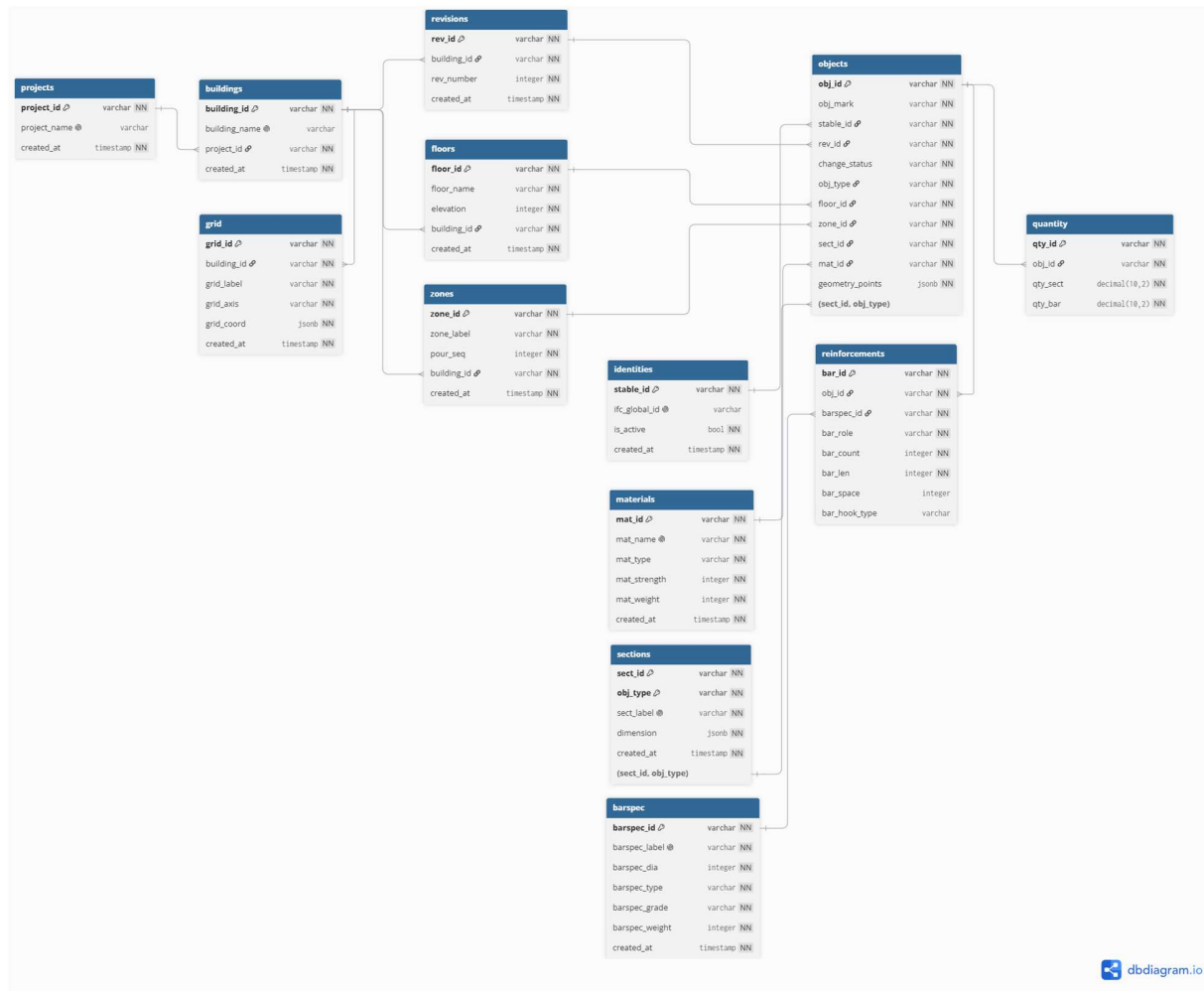
v0.3 → v0.4

- *sections* had no human-readable identifier; *sect_id* is a system-generated key not guaranteed to convey meaning; added required *sect_label*
- *zones* had the same gap; *zone_id* and *pour_seq* together still gave no readable way to identify a zone; added required *zone_label*

v0.4 → v1.0 (finalized ERD)

- *materials.mat_name* was optional with no uniqueness constraint, so it could not reliably function as a lookup key; changed to not null and unique
- *materials* was missing *mat_weight* (unit weight), needed for the quantity calculation; added
- *barspec* had no single human-readable label field; added *barspec_label* (unique), and was missing *barspec_weight*; added
- *projects.project_name* and *buildings.building_name* gained a global uniqueness constraint (remain nullable by design, falling back to displaying the row's own ID when empty)

- *floors.floor_name* and *zones.zone_label* gained a composite uniqueness constraint scoped to *building_id*, not global, since two different buildings may each have their own "Level 2"
- *sections.sect_label* gained a global uniqueness constraint so that assignment can address the contextual name instead of the database-facing ID
- *quantity.qty_sect* and *quantity.qty_bar* were integer, which cannot represent a usual BOQ-standard decimal value precisely; changed to decimal(10,2)
-



Finalized ERD

ii. Codebase Implementation

15 September 2026 — Codex Prompting for Design Implementation to Codebase (Phase i)

Context

Once the design was formed, I built a workflow for Codex to follow, based on the commit structure and the design decisions formed at the time, later updated in Phase 2. I uploaded the design decisions along with the ERD. Before that, though, I used AI to expand the workflow so commits would stay small and frequent.

Prompt (Verbatim, translated to English)

1. Clone the Week 2 repo
2. Create a branch from Week 2: feat/week-3-deliverables
3. Implement the changes we discussed, one commit at a time:
 - 3a. Commit 1: build models.py from the fields we've already worked out, including a file for change_status validation
 - 3b. Commit 2: Alembic migration (should this be separate from 3a, or combined? Check against Postgres once done)
 - 3b. Commit 3: wire it into the backend's main.py, auto-generate the types, test via /docs
 - 3c. Commit 4: build a POST endpoint for entering data — I still don't understand how to determine which data is human input and which gets written by the system
 - 3d. Commit 5: build the frontend(?)

Result

The resulting commits for the Codex workflow prompt were more detailed, as follows:

0. Setting up the GitHub repository clone
 1. Adding Building and Project in SQLAlchemy models, doing Alembic migration
 2. Renaming Storeys to Floors, adding building_id to Floor/Zone, adding new Grid table
 3. Renaming Dimension to Section, adding BarSpec table
 4. Adding Revision table with rev_number for linearity enforcement
 5. Adding Identity table
 6. Replacing Member with Object, restructuring around stable_id/rev_id
 7. Adding Reinforcement and Quantity tables
 8. Rewriting seed data for Project Configuration only
 9. Implementing the Pre-Validation Gate
 10. Implementing the Stable ID Assignment workflow
 11. Implementing the Change Status Determination workflow
 12. Implementing the Active Status Sync workflow
 13. Adding the Bulk Upload revision endpoint
 14. Adding the Interactive Edit revision endpoint
 15. Regenerating OpenAPI types, verifying endpoints via /docs

Judgement and Validations

Judgement is made by inspecting the finished codebase produced by Codex. This approach is taken so that focus stays on the design and on solving problems that surface along the way, instead of straying into coding everything from the ground up as in Week 2, where I eventually did not know whether the output was right or wrong. This way, I check whether the implementation matches the design, and if not, what needs to change, whether that is the codebase, the data model, or a mistaken design decision. From here, this also looped back into finalizing the model.

16 September 2026 — Codex Prompting for Design Implementation to Codebase (Phase II)

Context

After the first phase of the codebase was finished, I examined it against the ERD, the design decisions, and the problem-solving done for issues that had surfaced, and asked for a lot of fixes, requesting a cross-check against any revision of those two documents as well. Throughout the conversation with AI, the items added were as follows:

- Replace uuid4() ID generation with prefix-sequential, human-readable IDs (e.g. OBJ-001, RBAR-0001), matching the Data Dictionary convention. *[See decision D08]*
- Resolve a prefix collision between Reinforcement and BarSpec by assigning BAR- to BarSpec and RBAR- to Reinforcement. *[Cross-checked with AI's help]*
- Correct obj_type from the old 4-value abbreviated set to the full 6-value list (column, beam, wall, slab, footing, stair), per the source documents. *[Cross-checked with AI's help]*
- Add previously missing label fields (sect_label, zone_label, mat_weight, barspec_label, barspec_weight) and their corresponding uniqueness constraints, per the final Data Dictionary. *[See decision D07]*
- Fix seed data: correct grid_coord from an implicit scalar to an explicit {x, y} object. *[So that grid axis generation is not implicit]*
- Add validation support for circular sections. *[Part of the pre-validation gate's contextual completeness check]*
- Rework ObjectInput to accept human-readable names (floor_name, zone_label, sect_label, mat_name) instead of raw database IDs, with server-side resolution to the underlying ID. *[See decision D11]*
- Restructure endpoints to be fully nested (/projects/{id}/buildings/{id}/...), replacing the building-only-scoped routes *[So that every layer of the hierarchy is visible and multiple projects can be browsed]*
- Add POST endpoints for all Project Configuration entities (Project, Building, Floor, Zone, Material, Section, BarSpec, Grid), enabling setup via user input rather than seed scripts alone *[See decision D10]*
- Add nested GET endpoints (Project → Building → Revision → Object) for hierarchical browsing. *[Initially there was no frontend for this; changed to read-only, to make it easier to fill in the JSON body for data input as well]*
- Implement the Quantity calculation formula (qty_sect, qty_bar), which had not yet been built — including the concrete (m³) vs. steel (kg) output distinction. *[Simple algorithm that computes section area or thickness multiplied by length, or polygon area, based on the geometry point]*
- Enforce reinforcement cardinality by material type: concrete objects require at least one reinforcement row, steel objects may have none. *[validation so that steel structures are not required to carry reinforcement, following engineering logic]*
- Add a read-only frontend browsing view (matching Week 2 styling), for data display only, not input/editing *[To make it easier to check and test input as well]*

Result

The resulting commits for the Codex workflow prompt were more detailed, continuing from previous 15 commits in phase 1 codex work as follows:

16. Replacing uuid4() with sequential, prefixed IDs
17. Correcting obj_type to full names, adding footing and stair
18. Adding label/weight fields and uniqueness constraints

19. Fixing seed grid_coord format, adding circular section validation
20. Resolving Object input by natural name instead of raw ID
21. Nesting revision endpoints under project and building
22. Adding POST endpoints for Project Configuration
23. Adding GET endpoints for hierarchy browsing
24. Implementing quantity calculation
25. Enforcing reinforcement cardinality by material type
26. Adding frontend project/building/revision browsing

Judgement and Validation

Judgement is made the same way as before: by inspecting the finished codebase produced by Codex, checking whether the implementation matches the design and, where it does not, whether the fix belongs in the codebase, the data model, or a mistaken design decision.

This was run in Codex, and I read through and tested it; there were still a few things I was not satisfied with, which became the content of commits 16 through completion, along with the items I rejected after commit 26.

The following are the things I checked and revised after the second phase of codex generation

- Add GET endpoints for all project config, add a project config page *[For browsing the configuration catalogue, so that matching an attribute name to an object is easier when entering input]*
- Wipe all existing/old seed.py data so that data input purely comes from the JSON funnel *[To safeguard input against corrupted data that does not follow the designed convention, decision D08]*
- Fix a bug on the service model python: the carry forward logic for unchanged objects failed the active status sync workflow as described in D18, causing a deleted member in revision 2 to be explicitly set to “deleted” state, yet remain present with status “unchanged” in revision 3 when it is supposedly omitted from the table *[Done after testing and reading the service model code]*
- Sense-checking: reviewed the POST endpoint definitions in the backend model, which require the user to type in the database-facing *_id to choose the project and building where the object should be entered, instead of the contextual label as designed in D11. Confirmed this was a limitation of the current stage, since the mechanism that would automatically route to the correct API address through UI navigation is not yet built; noted as a limitation in the acknowledgement section.

By this point, the repository can already run the full cycle of data from input, revision, deletion, and producing quantity. However, the data funnel is currently still handled mostly through /docs so as not to expand the deliverable scope into UI design: data input is handled via POST endpoints without a UI form, the frontend is used solely for field browsing and read-only purposes, and data management such as renaming and deleting still relies on direct *psql* operations.

16 September 2026 — Input Architecture Test Cases: Pre-Validation Gates and “Added” Status

Context

Once all commits were running, I tested by entering 4 members through the bulk upload POST endpoint. The expected result on the frontend should be 4 new additional member entries under Revision 0, all with

an “added” change status, along with the correct attributes as entered in the endpoint. AI was used to auto-complete the JSON body entry.

Prompt (paraphrased in English)

Make a JSON body for 4 columns object addition in each corner of grid, with 4D16; D13-150 reinforcement

Result

Initial Data Entry (Revision 0)

```
{
  "objects": [
    {
      "is_new": true,
      "obj_mark": "C1.F01.001",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
      "sect_label": "C1 - 400x400 column",
      "mat_name": "concrete K250",
      "geometry_points": [[0, 0, 0], [0, 0, 3500]],
      "reinforcements": [
        {
          "barspec_label": "D16 deformed BjTS 420",
          "bar_role": "longitudinal",
          "bar_count": 4,
          "bar_len": 3000,
          "bar_space": null,
          "bar_hook_type": null
        },
        {
          "barspec_label": "D13 deformed BjTS 420",
          "bar_role": "transverse",
          "bar_count": 21,
          "bar_len": 1600,
          "bar_space": 150,
          "bar_hook_type": "135-degree hook"
        }
      ]
    },
    {
      "is_new": true,
      "obj_mark": "C1.F01.002",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
      "sect_label": "C1 - 400x400 column",
      "mat_name": "concrete K250",
      "geometry_points": [[8000, 0, 0], [8000, 0, 3500]],
      "reinforcements": [
        {
          "barspec_label": "D16 deformed BjTS 420",
          "bar_role": "longitudinal",
          "bar_count": 4,
          "bar_len": 3000,
          "bar_space": null,
          "bar_hook_type": null
        },
        {
          "barspec_label": "D13 deformed BjTS 420",
          "bar_role": "transverse",
          "bar_count": 21,
          "bar_len": 1600,
          "bar_space": 150,
          "bar_hook_type": "135-degree hook"
        }
      ]
    }
  ]
}
```

```

{
  "is_new": true,
  "obj_mark": "C1.F01.003",
  "obj_type": "column",
  "floor_name": "ground floor",
  "zone_label": "Zone 1",
  "sect_label": "C1 - 400x400 column",
  "mat_name": "concrete K250",
  "geometry_points": [[0, 6000, 0], [0, 6000, 3500]],
  "reinforcements": [
    {
      "barspec_label": "D16 deformed BjTS 420",
      "bar_role": "longitudinal",
      "bar_count": 4,
      "bar_len": 3000,
      "bar_space": null,
      "bar_hook_type": null
    },
    {
      "barspec_label": "D13 deformed BjTS 420",
      "bar_role": "transverse",
      "bar_count": 21,
      "bar_len": 1600,
      "bar_space": 150,
      "bar_hook_type": "135-degree hook"
    }
  ]
},
{
  "is_new": true,
  "obj_mark": "C1.F01.004",
  "obj_type": "column",
  "floor_name": "ground floor",
  "zone_label": "Zone 1",
  "sect_label": "C1 - 400x400 column",
  "mat_name": "concrete K250",
  "geometry_points": [[8000, 6000, 0], [8000, 6000, 3500]],
  "reinforcements": [
    {
      "barspec_label": "D16 deformed BjTS 420",
      "bar_role": "longitudinal",
      "bar_count": 4,
      "bar_len": 3000,
      "bar_space": null,
      "bar_hook_type": null
    },
    {
      "barspec_label": "D13 deformed BjTS 420",
      "bar_role": "transverse",
      "bar_count": 21,
      "bar_len": 1600,
      "bar_space": 150,
      "bar_hook_type": "135-degree hook"
    }
  ]
}
]

```

Judgement and Validation

The result matched the expected result, showing 4 new entries under the auto-generated revision numbering (Rev 0), all with an “added” change status. I then also tried submitting a JSON body with an attribute that does not exist in the configuration, which, as expected, returned an error message.

ObjectsProject Configuration

AISIMS Objects

Browse objects across a project's buildings and revisions. Creation and editing happen via <http://localhost:8000/docs> - this view is read-only.

PROJECTBUILDINGREVISION

Pilot Project 001Tower ARev 0

Objects4 total

MEMBER ID	STABLE ID	TYPE	FLOOR
C1.F01.001	538b658a-b6d9-4f84-a18a-823b8549327e	Column	ground floor
C1.F01.002	2b462bf6-3c5a-411d-b67c-8e889864f2b	Column	ground floor
C1.F01.003	c97a4a1f-dbcd-448c-95c8-7e888348eb6	Column	ground floor
C1.F01.004	2e928aca-07cd-49d8-a53f-2f583388e48	Column	ground floor

Object Detail

C1.F01.001ColumnAdded

Stable ID

538b658a-b6d9-4f84-a18a-823b8549327e

Floor

ground floor

Zone

Zone 1

Section

C1 - 400x400 column

Material

concrete K250

QTY. SECTION

0.56 m²/kg

QTY. BAR

53.95 kg

Rev 0 Initial Entries Table

PROJECTBUILDINGREVISION

Pilot Project 001Tower ARev 0

Objects4 total

FLOOR	ZONE	SECTION	MATERIAL	STATUS
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added
ground floor	Zone 1	C1 - 400x400 column	concrete K250	Added

Rev 0 Change Status

Page 22

Object Detail

C1.F01.001

Column

Added

Stable ID

530b650a-b6d9-4f84-a10a-823b8549327e

Floor

ground floor

Zone

Zone 1

Section

C1 - 400x400 column

Material

concrete K250

QTY. SECTION

0.56 m²/kg

QTY. BAR

53.95 kg

SECTION DIMENSION

Depth

400

Shape

rectangular

Width

400

GEOMETRY POINTS

POINT	X (MM)	Y (MM)	Z (MM)
P1	0	0	0
P2	0	0	3500

REINFORCEMENT

BAR SPEC	DIA (MM)	GRADE
D16 deformed BjTS 420	16	BjTS 420
D13 deformed BjTS 420	13	BjTS 420

Object Details (Error still apparent on the quantity unit, later fixed)

16 September 2026 —Revision Workflow Test Cases: Change Status

Context

Now testing the edit endpoint, entering changes to 2 columns with is_new=false and 1 added column with is_new=true (Rev 1). The expected result should return 5 columns, since the two remaining columns are untouched and should be carried forward; the change status should show 2 modified columns and 1 added column, along with the correct attributes as entered in the endpoint.

Prompt (paraphrased in English)

Make a JSON body for 2 column change and 1 additional column under the edit revision POST endpoint.

Result

Triggering Revision 1: 2 Column Change and 1 Added Column

```
{
  "changed_objects": [
    {
      "is_new": true,
      "obj_mark": "C3.F01.005",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
      "sect_label": "C3 - 300x300 column",
      "mat_name": "concrete K250",
      "geometry_points": [[4000, 0, 0], [4000, 0, 3500]],
      "reinforcements": [
        {
          "barspec_label": "D16 deformed BjTS 420",
          "bar_role": "longitudinal",
          "bar_count": 4,
          "bar_len": 3000,
          "bar_space": null,
          "bar_hook_type": null
        },
        {
          "barspec_label": "D13 deformed BjTS 420",
          "bar_role": "transverse",
          "bar_count": 21,
          "bar_len": 1200,
          "bar_space": 150,
          "bar_hook_type": "135-degree hook"
        }
      ]
    },
    {
      "is_new": false,
      "stable_id": "c97a4a1f-dbcd-440c-95c6-7e08d0348eb6",
      "obj_mark": "C2.F01.003",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
      "sect_label": "C2 - \u00d8350 column",
      "mat_name": "concrete K250",
      "geometry_points": [[0, 6000, 0], [0, 6000, 3500]],
      "reinforcements": [
        {
          "barspec_label": "D16 deformed BjTS 420",
          "bar_role": "longitudinal",
          "bar_count": 4,
          "bar_len": 3000,
          "bar_space": null,
          "bar_hook_type": null
        },
        {
          "barspec_label": "D13 deformed BjTS 420",
          "bar_role": "transverse",
          "bar_count": 21,
          "bar_len": 1600,
          "bar_space": 150,
          "bar_hook_type": "135-degree hook"
        }
      ]
    },
    {
      "is_new": false,
      "stable_id": "2e928aca-07cd-49d8-a53f-2f5833808e48",
      "obj_mark": "C2.F01.004",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
    }
  ]
}
```

```

"sect_label": "C2 - \u00d8350 column",
"mat_name": "concrete K250",
"geometry_points": [[8000, 6000, 0], [8000, 6000, 3500]],
"reinforcements": [
  {
    "barspec_label": "D16 deformed BjTS 420",
    "bar_role": "longitudinal",
    "bar_count": 4,
    "bar_len": 3000,
    "bar_space": null,
    "bar_hook_type": null
  },
  {
    "barspec_label": "D13 deformed BjTS 420",
    "bar_role": "transverse",
    "bar_count": 21,
    "bar_len": 1600,
    "bar_space": 150,
    "bar_hook_type": "135-degree hook"
  }
]
},
"deleted_stable_ids": []

```

Judgement and Validation

The result matched the expected result.

The screenshot displays the AISIMS Objects web application. At the top, there are tabs for 'Objects' and 'Project Configuration'. Below the title 'AISIMS Objects', a subtitle reads: 'Browse objects across a project's buildings and revisions. Creation and editing happen via <http://localhost:8000/docs> - this view is read-only.'

Filters for PROJECT, BUILDING, and REVISION are set to 'Pilot Project 001', 'Tower A', and 'Rev 1' respectively. The main table, titled 'Objects' with a '5 total' count, lists the following entries:

MEMBER ID	STABLE ID	TYPE	FLOOR
C1.F01.001	538b658a-b6d9-4f84-a19a-823b8549327e	Column	ground floor
C1.F01.002	2b462bf6-3c3a-411d-b67c-8eb88964f2b	Column	ground floor
C2.F01.003	c97a4a1f-dbcd-448c-95c6-7e88d8348eb6	Column	ground floor
C2.F01.004	2e928aca-87cd-49d8-a53f-2f5833888e48	Column	ground floor
C3.F01.005	ed3939a3-d9d9-4e41-ad32-5e485f86e9b8	Column	ground floor

The 'Object Detail' panel on the right shows details for 'C1.F01.001', which is a 'Column' and is 'Unchanged'. The details include:

- Stable ID: 538b658a-b6d9-4f84-a19a-823b8549327e
- Floor: ground floor
- Zone: Zone 1
- Section: C1 - 480x480 column
- Material: concrete K250
- QTY. SECTION: 0.56 m³
- QTY. BAR: 53.95 kg

At the bottom of the detail panel, it says 'SECTION DIMENSION'.

Rev 1 Entries Table

PROJECT

BUILDING

REVISION

Pilot Project 001

Tower A

Rev 1

Objects

5 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Modified
und floor	Zone 1	C2 - Ø350 column	concrete K250	Modified
und floor	Zone 1	C3 - 300x300 column	concrete K250	Added

Rev 1 Change Status

16 September 2026 — Active Sync Workflow Test Cases: Deletion and Restoration

Context

Now testing the edit endpoint by deleting one member (Rev 2); AI was not used. The expected result is that the table should still have 5 entries: 4 columns unchanged and 1 deleted, along with the correct attributes as entered in the endpoint. The deletion is done by submitting the stable ID.

Triggering Revision 2: Deleting a Mamber

```
{
  "deleted_stable_ids": [ed3939a3-d9d8-4e41-ada2-5e485f86e9b8]
}
```

Followed by testing the active sync workflow, by triggering a revision (Rev 3) without changing any entries. The expected result should be 4 entries, because of the object deleted in the previous revision; the remaining, untouched objects should stay in “unchanged” status. AI was not used.

Triggering Revision 3: Unchanged Objects

```
{
  "changed_objects": []
}
```

Followed by testing the restoration of a previously deleted member (Rev 4), using its registered stable_id in the identity table with an is_new=false entry. This shows the importance of having a separate identity entity that stores every registered object, which is what makes restoration possible. The expected result should be 5 entries, because of the restored object; the remaining, untouched objects should stay in “unchanged” status. AI was used to autofill the JSON body.

Triggering Revision 4: Restoring Deleted Object

```
{
  "changed_objects": [
    {
      "is_new": false,
      "stable_id": "ed3939a3-d9d8-4e41-ada2-5e485f86e9b8",
      "obj_mark": "C3.F01.005",
      "obj_type": "column",
      "floor_name": "ground floor",
      "zone_label": "Zone 1",
      "sect_label": "C3 - 300x300 column",
      "mat_name": "concrete K250",
      "geometry_points": [[4000, 0, 0], [4000, 0, 3500]],
      "reinforcements": [
        {
          "barspec_label": "D16 deformed BjTS 420",
          "bar_role": "longitudinal",
          "bar_count": 4,
          "bar_len": 3000,
          "bar_space": null,
          "bar_hook_type": null
        },
        {
          "barspec_label": "D13 deformed BjTS 420",
          "bar_role": "transverse",
          "bar_count": 21,
          "bar_len": 1200,
          "bar_space": 150,
          "bar_hook_type": "135-degree hook"
        }
      ]
    }
  ],
  "deleted_stable_ids": []
}
```

Judgement and Validation

Revision 2 matched as expected.

PROJECT

Pilot Project 001

▼

BUILDING

Tower A

▼

REVISION

Rev 2

▼

Objects

5 total

OR	ZONE	SECTION	MATERIAL	STATUS
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C1 - 400x400 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C2 - Ø350 column	concrete K250	Unchanged
und floor	Zone 1	C3 - 300x300 column	concrete K250	Deleted

Rev 2 Change Status

Revision 3 initially turned up a bug where the member deleted in Revision 2 was still carried forward into Revision 3 with an “unchanged” status, when it should have been gone. The root cause was that the active sync workflow was only active on the bulk upload endpoint, while the interactive edit endpoint should carry the same logic. The model confused carrying forward an untouched member in an interactive edit with leaving out the active sync logic, when active sync should have taken priority in the validation check. The bug fix was to add active-status sync to the revision service, so that object forwarding is active-status aware for the interactive edit endpoint as well. After the bug fix, Revision 3 matched as expected, returning only 4 entries.

Objects

Project Configuration

AISIMS Objects

Browse objects across a project's buildings and revisions. Creation and editing happen via <http://localhost:8000/docs> - this view is read-only.

PROJECT

BUILDING

REVISION

Pilot Project 001

Tower A

Rev 3

Objects

4 total

MEMBER ID	STABLE ID	TYPE	FLOOR
C1_F01_001	538b658a-b6d9-4f84-a18a-823b8549327e	Column	ground floor
C1_F01_002	2b462bf6-3c5a-411d-b67c-9eb889864f2b	Column	ground floor
C2_F01_003	c97a4a1f-dbcd-44bc-95c6-7e8d8348eb6	Column	ground floor
C2_F01_004	2e928aca-87cd-49d8-a53f-2f5833888e48	Column	ground floor

Object Detail

Select an object from the table.

Rev 3 Entries Table (After bug fix)

Revision 4 matched as expected, showing the restored object.

ObjectsProject Configuration

AISIMS Objects

Browse objects across a project's buildings and revisions. Creation and editing happen via <http://localhost:8000/docs> - this view is read-only.

PROJECT

Pilot Project 001

BUILDING

Tower A

REVISION

Rev 4

Objects

5 total

MEMBER ID	STABLE ID	TYPE	FLOOR
C1.F01.001	538b658a-b6d9-4f84-a10a-823b8549327e	Column	ground floor
C1.F01.002	2b462bf6-3c5a-411d-b67c-9eb889964f2b	Column	ground floor
C2.F01.003	c97a4a1f-dbcd-449c-95c0-7e88d8348eb6	Column	ground floor
C2.F01.004	2e928aca-07cd-49d8-a53f-2f5833889e48	Column	ground floor
C3.F01.005	ed3939a3-d9d8-4e41-ada2-5e485f86e9b8	Column	ground floor

Object Detail

C3.F01.005 • ColumnRestored

Stable ID

ed3939a3-d9d8-4e41-ada2-5e485f86e9b8

Floor

ground floor

Zone

Zone 1

Section

C3 - 300x300 column

Material

concrete K250

QTY. SECTION

0.32 m³/kg

QTY. BAR

45.20 kg

SECTION DIMENSION

Rev 2 Entries Table (Restored object shown)

19-21 September 2026 — Written Deliverables Wrap Up